
● SAVOL 56:

[PANDAS + SEABORN] OCHIQ DATASETDAN FOYDALANIB: NaN QIYMATLARNI TOPING VA TO'LDIRING, KATEGORIK USTUN UCHUN VALUE_COUNTS() CHIQARING VA NATIJANI BAR CHART DA KO'RSATING.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 1. DATASET YUKLASH
# =====

# Titanic – mashhur ochiq dataset
# 891 ta yo'lovchi, 12 ta ustun
# NaN qiymatlari bor: Age, Cabin, Embarked
# Kategorik ustunlar: Sex, Pclass, Embarked, Survived
#
# Ustunlar:
# Survived → 0=halok, 1=omon qoldi
# Pclass → bilek sinfi (1=birinchi, 2=ikkinchi, 3=uchinchi)
# Sex → jins
# Age → yosh
# SibSp → aka-uka/opa-singillar soni
# Parch → ota-ona/bolalar soni
# Fare → bilek narxi
# Embarked → jo'nash porti (C=Cherbourg, Q=Queenstown, S=Southampton)

url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

try:
    df = pd.read_csv(url)
    print(" Internet orqali yuklandi ✓")
except Exception:
    # Internet bo'lmasa – qo'lda yaratamiz
    print(" Lokal dataset yaratilmoqda...")
    np.random.seed(42)
    n = 891
    df = pd.DataFrame({
        "PassengerId": range(1, n + 1),
        "Survived": np.random.choice([0, 1], n, p=[0.62, 0.38]),
        "Pclass": np.random.choice([1, 2, 3], n, p=[0.24, 0.21, 0.55]),
        "Name": [f"Passenger_{i}" for i in range(n)],
        "Sex": np.random.choice(
            ["male", "female"], n, p=[0.65, 0.35]),
        "Age": np.where(
            np.random.rand(n) < 0.20,
            np.nan,
            np.random.uniform(1, 80, n).round(1)),
        "SibSp": np.random.choice([0, 1, 2, 3], n, p=[0.68, 0.23, 0.07, 0.02]),
```

```

"Parch":      np.random.choice([0,1,2,3], n, p=[0.76,0.13,0.09,0.02]),
"Ticket":     [f"T{i:05d}" for i in range(n)],
"Fare":       np.random.exponential(33, n).round(2),
"Cabin":      np.where(
    np.random.rand(n) < 0.77,
    np.nan,
    np.random.choice(
        ["A1","B2","C3","D4","E5"], n)),
"Embarked":   np.where(
    np.random.rand(n) < 0.002,
    np.nan,
    np.random.choice(
        ["S","C","Q"], n, p=[0.72,0.19,0.09])),
    })

print("=" * 56)
print("  TITANIC DATASET:")
print("=" * 56)
print(f"  Jami namuna : {df.shape[0]:,} ta")
print(f"  Ustunlar    : {df.shape[1]} ta")
print(f"\n  Dastlabki 3 qator:")
print(df[["Survived","Pclass","Sex","Age",
    "Fare","Embarked"]].head(3).to_string())

# =====
# 2. NaN QIYMATLARNI TOPISH
# =====

# isnull() → har katak uchun True/False (NaN bo'lsa True)
# .sum()   → ustun bo'ylab True larni sanash = NaN soni
# [mask]   → faqat NaN bor ustunlarni ko'rsatish
nan_soni   = df.isnull().sum()
nan_bor    = nan_soni[nan_soni > 0]

# NaN foizi → qanchasi yo'q
nan_foiz   = (nan_bor / len(df) * 100).round(2)

print("\n" + "=" * 56)
print("  NaN QIYMATLAR (TO'LDIRISHDAN OLDIN):")
print("=" * 56)
print(f"  {'Ustun':<12} {'NaN soni':>10} {'Foiz':>10}")
print("  " + "-" * 34)
for ustun in nan_bor.index:
    print(f"  {'ustun':<12} {'nan_bor[ustun]:>10} "
        f"{'nan_foiz[ustun]:>9.1f}%")

# =====
# 3. NaN QIYMATLARNI TO'LDIRISH
# =====

df_toza = df.copy() # Asl datasetni o'zgartirmaslik uchun nusxa

# — Age → median bilan to'ldirish —————
# Nima uchun median?
# mean (o'rtacha) → chekka qiymatlar (outlier) ta'sirida buziladi
# median          → barqaror, chekka qiymatlar ta'sir qilmaydi
yosh_median = df_toza["Age"].median()

```

```

df_toza["Age"] = df_toza["Age"].fillna(yosh_median)

print("\n" + "=" * 56)
print("  NaN TO'LDIRISH STRATEGIYALARI:")
print("=" * 56)
print(f"   Age      → median bilan to'ldirildi"
      f" (median = {yosh_median:.1f})")

# — Embarked → mode (eng ko'p uchraydigan) bilan —————
# Kategorik ustun → o'rtacha hisoblab bo'lmaydi
# mode() → eng ko'p uchraydigan qiymat (modal qiymat)
# [0] → mode() Series qaytaradi → birinchi elementni olish
embarked_mode = df_toza["Embarked"].mode()[0]
df_toza["Embarked"] = df_toza["Embarked"].fillna(embarked_mode)
print(f"   Embarked → mode bilan to'ldirildi"
      f" (mode = '{embarked_mode}')")

# — Cabin → "Noma'lum" bilan to'ldirish —————
# Cabin 77% bo'sh → statistik to'ldirish mantiqsiz
# "Noma'lum" kategoriyasi yaratiladi
df_toza["Cabin"] = df_toza["Cabin"].fillna("Noma'lum")
print(f"   Cabin   → \"Noma'lum\" bilan to'ldirildi"
      f" (77% bo'sh edi)")

# — Tekshirish —————
nan_qoldi = df_toza.isnull().sum().sum()
print(f"\n   Qolgan NaN soni: {nan_qoldi} ta "
      f"{'✓ Tozalandi!' if nan_qoldi == 0 else '⚠ Hali bor'}")

# =====
# 4. VALUE_COUNTS – KATEGORIK USTUNLAR
# =====

# value_counts() → har noyob qiymat necha marta uchraganini sanaydi
# normalize=True → sanoq o'rniga nisbat (foiz) chiqaradi

kategorik = ["Pclass", "Sex", "Embarked", "Survived"]

print("\n" + "=" * 56)
print("  VALUE_COUNTS – KATEGORIK USTUNLAR:")
print("=" * 56)

for ustun in kategorik:
    sanoq = df_toza[ustun].value_counts()
    foiz = df_toza[ustun].value_counts(normalize=True) * 100

    print(f"\n   [{ustun}]")
    print(f"   {'Qiymat':<15} {'Soni':>8} {'Foiz':>8}")
    print("   " + "-" * 33)
    for qiymat in sanoq.index:
        print(f"   {str(qiymat):<15} "
              f"{sanoq[qiymat]:>8} "
              f"{foiz[qiymat]:>7.1f}%")

# =====
# 5. VIZUALIZATSIYA – 4 ta bar chart
# =====

```

```

plt.figure(figsize=(15, 10), facecolor="#0f1117")
plt.suptitle(
    "Titanic Dataset – Kategorik Ustunlar Tahlili",
    fontsize=16, fontweight="bold",
    color="#e2e8f0", y=0.98,
)

# Seaborn mavzusi
sns.set_theme(style="dark", rc={
    "axes.facecolor": "#1a1d27",
    "figure.facecolor": "#0f1117",
    "text.color": "#cdd6f4",
    "axes.labelcolor": "#8892b0",
    "xtick.color": "#8892b0",
    "ytick.color": "#8892b0",
    "axes.edgecolor": "#2d3148",
    "grid.color": "#2d3148",
})

# Har grafik uchun rang palitralari
palitalar = [
    ["#7c85f3", "#f38ba8", "#a6e3a1"], # Pclass
    ["#7c85f3", "#f38ba8"], # Sex
    ["#a6e3a1", "#fab387", "#89dceb"], # Embarked
    ["#f38ba8", "#a6e3a1"], # Survived
]

# Har grafik uchun o'q nomlari va sarlavhalar
meta = {
    "Pclass": {
        "sarlavha": "Bilet Sinfi (Pclass)",
        "xlabel": "Sinf",
        "ylabel": "Yo'lovchilar soni",
        "xticklabel": {1: "1-sinf\n(Yuqori)",
                        2: "2-sinf\n(O'rta)",
                        3: "3-sinf\n(Quyi)"},
    },
    "Sex": {
        "sarlavha": "Jins (Sex)",
        "xlabel": "Jins",
        "ylabel": "Yo'lovchilar soni",
        "xticklabel": {"male": "Erkak", "female": "Ayol"},
    },
    "Embarked": {
        "sarlavha": "Jo'nash Porti (Embarked)",
        "xlabel": "Port",
        "ylabel": "Yo'lovchilar soni",
        "xticklabel": {"S": "Southampton\n(S)",
                        "C": "Cherbourg\n(C)",
                        "Q": "Queenstown\n(Q)"},
    },
    "Survived": {
        "sarlavha": "Omon Qolish (Survived)",
        "xlabel": "Natija",
        "ylabel": "Yo'lovchilar soni",
        "xticklabel": {0: "Halok bo'ldi\n(0)",

```

```

        1: "Omon qoldi\n(1)"}},
    },
}

for i, (ustun, palitra) in enumerate(
    zip(kategorik, palitalar), start=1):

    ax = plt.subplot(2, 2, i)

    sanoq      = df_toza[ustun].value_counts().sort_index()
    qiymatlar  = list(sanoq.index)
    sonlar     = list(sanoq.values)

    # seaborn barplot
    bars = sns.barplot(
        x      = qiymatlar,
        y      = sonlar,
        palette= palitra[:len(qiymatlar)],
        ax     = ax,
        width  = 0.55,
        edgecolor="#0f1117",
        linewidth=0.8,
    )

    # Har ustun tepasiga son + foiz yozish
    jami = sum(sonlar)
    for j, (son, patch) in enumerate(
        zip(sonlar, ax.patches)):
        foiz = son / jami * 100
        ax.text(
            patch.get_x() + patch.get_width() / 2,
            patch.get_height() + jami * 0.012,
            f"{son:},\n({foiz:.1f}%)",
            ha      = "center",
            va      = "bottom",
            color    = "#e2e8f0",
            fontsize = 10,
            fontweight= "bold",
        )

    # O'q belgilarini matnli nom bilan almashtirish
    m = meta[ustun]
    ax.set_xticks(range(len(qiymatlar)))
    ax.set_xticklabels(
        [m["xticklabel"].get(q, str(q)) for q in qiymatlar],
        fontsize=10,
    )

    # Y o'qi yuqori chegarasini kengaytirish (yozuv sig'sin)
    ax.set_ylim(0, max(sonlar) * 1.22)

    ax.set_title(
        m["sarlavha"],
        color="#e2e8f0", fontsize=13,
        fontweight="bold", pad=12,
    )
    ax.set_xlabel(m["xlabel"], labelpad=8, fontsize=10)

```

```

ax.set_ylabel(m["ylabel"], labelpad=8, fontsize=10)
ax.grid(
    axis="y", color="#2d3148",
    linestyle="--", alpha=0.6, zorder=0,
)
for spine in ax.spines.values():
    spine.set_edgecolor("#2d3148")

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Chiqish Natijasi

```

=====
TITANIC DATASET:
=====

Jami namuna : 891 ta
Ustunlar    : 12 ta

=====
NaN QIYMATLAR (TO'LDIRISHDAN OLDIN):
=====

```

Ustun	NaN soni	Foiz
Age	177	19.9%
Cabin	687	77.1%
Embarked	2	0.2%

```

=====
NaN TO'LDIRISH STRATEGIYALARI:
=====

Age      → median bilan to'ldirildi (median = 28.0)
Embarked → mode bilan to'ldirildi (mode = 'S')
Cabin    → "Noma'lum" bilan to'ldirildi (77% bo'sh edi)

Qolgan NaN soni: 0 ta ✓ Tozalandi!

=====
VALUE_COUNTS – KATEGORIK USTUNLAR:
=====

```

[Pclass]		
Qiymat	Soni	Foiz
1	216	24.2%
2	184	20.7%
3	491	55.1%

[Sex]		
Qiymat	Soni	Foiz
male	577	64.8%
female	314	35.2%

```

[Embarked]

```

Qiymat	Soni	Foiz
S	646	72.5%
C	168	18.9%
Q	77	8.6%

[Survived]		
Qiymat	Soni	Foiz
0	549	61.6%
1	342	38.4%

Dastur Tuzilishi

```

Dastur
├── Dataset yuklash
│   └── pd.read_csv(url)          → Titanic (891×12)
├── NaN topish
│   └── isnull().sum()           → ustun bo'ylab sanoq
├── NaN to'ldirish
│   ├── Age      → fillna(median) → raqamli, barqaror
│   ├── Embarked→ fillna(mode)   → kategorik
│   └── Cabin    → fillna("Noma'lum") → 77% bo'sh
├── value_counts()
│   └── har kategorik ustun      → soni + foizi
└── Vizualizatsiya
    └── sns.barplot()            → 4 ta subplot (2×2)

```

Asosiy Texnik Tushunchalar

NaN TO'LDIRISH STRATEGIYALARI:

Raqamli ustun:

```

fillna(mean)    → tez, lekin outlier ta'sir qiladi
fillna(median)  → barqaror, outlier ta'sir qilmaydi ✓
fillna(0)       → ma'nosi bo'lsa (masalan: soni)

```

Kategorik ustun:

```

fillna(mode)    → eng ko'p uchraydigan qiymat ✓
fillna("other") → alohida kategoriya

```

Ko'p bo'sh (>50%):

```

fillna("Noma'lum") → ma'lumot yo'qligini belgilash ✓
ustunni o'chirish  → drop(columns=["Cabin"])

```

Ilg'or usullar:

```

KNNImputer      → qo'shni namunalar asosida
IterativeImputer→ boshqa ustunlar asosida bashorat

```

NaN TOPISH USULLARI:

```

df.isnull().sum()      → ustun bo'ylab sanoq
df.isnull().sum().sum() → jami NaN soni

```

```
df.isnull().mean() * 100    → foiz sifatida
df[df["Age"].isnull()]      → NaN qatorlarni ko'rish
df.info()                  → har ustun uchun non-null soni
```

VALUE_COUNTS:

```
df["Sex"].value_counts()
→ male      577
→ female    314

df["Sex"].value_counts(normalize=True)
→ male      0.648    ← foiz (0-1)
→ female    0.352

df["Sex"].value_counts(dropna=False)
→ NaN ham hisobga olinadi
```

💡 **Eslab qol:** NaN to'ldirishda `fillna()` joyida o'zgartirmaydi — `df["Age"] = df["Age"].fillna(28)` yoki `inplace=True` parametri ishlatilishi **shart**. Shuningdek, to'ldirish **train dataseti** statistikasidan olinishi kerak — test ma'lumotlaridan median/mode hisoblash **data leakage** hisoblanadi va modelning haqiqiy samaradorligini oshirib ko'rsatadi.

● SAVOL 57:

[K-MEANS] NUMPY YORDAMIDA 2 O'LCHAMLI TASODIFIY 100 TA NUQTA GENERATSIYA QILING, K-MEANS BILAN 3 TA KLASTERG A AJRATING VA HAR KLASTERNI BOSHQA RANGDA SCATTER PLOT DA KO'RSATING.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# =====
# 1. MA'LUMOT GENERATSIYA QILISH
# =====

# random.seed() → takrorlanuvchi natija uchun
# Har safar bir xil "tasodifiy" sonlar chiqadi
np.random.seed(42)

# 3 ta grupp atrofida nuqtalar generatsiya qilamiz
# Haqiqiy klasterlar bor → K-means ularni topishi kerak
#
# np.random.randn(n, 2) → normal taqsimlangan n×2 massiv
# * tarqalish           → standart og'ishni kengaytirish
# + markaz              → siljitish (shift)

n_guruh = 35    # Har gruppadan nuqtalar soni (35×3 = 105 ≈ 100)
```



```

# Guruh 1: chap-pastki (markaz: -3, -3)
guruh1 = np.random.randn(n_guruh, 2) * 0.9 + np.array([-3, -3])

# Guruh 2: o'ng-pastki (markaz: +3, -2)
guruh2 = np.random.randn(n_guruh, 2) * 1.0 + np.array([3, -2])

# Guruh 3: yuqori-markaz (markaz: 0, +4)
guruh3 = np.random.randn(n_guruh, 2) * 0.8 + np.array([0, 4])

# np.vstack() → vertikal birlashtirish (ustma-ust qo'yish)
# (35,2) + (35,2) + (35,2) → (105, 2)
X = np.vstack([guruh1, guruh2, guruh3])

# Aralashtirib yuborish → K-means uchun qiyinroq bo'lsin
indekslar = np.random.permutation(len(X))
X = X[indekslar]

print("=" * 52)
print("  MA'LUMOT GENERATSIYASI:")
print("=" * 52)
print(f"  Jami nuqtalar : {len(X)} ta")
print(f"  O'lchamlar    : {X.shape[1]} ta (X, Y)")
print(f"  X oralig'i    : [{X[:,0].min():.2f}, {X[:,0].max():.2f}]")
print(f"  Y oralig'i    : [{X[:,1].min():.2f}, {X[:,1].max():.2f}]")

# =====
# 2. K-MEANS MODELI
# =====

K = 3  # Klasterlar soni

# n_clusters → nechta klaster topilsin
# init       → boshlang'ich markazlar tanlash usuli
#           "k-means++" → aqlli tanlash (tezroq, yaxshiroq)
#           "random"    → tasodifiy tanlash
# n_init     → necha marta qayta ishga tushirish (eng yaxshisi saqlanadi)
#           Ko'p → sekinroq, lekin global minimum topiladi
# max_iter   → bir ishga tushirishda maksimum iteratsiya
# random_state → takrorlanuvchi natija
kmeans = KMeans(
    n_clusters = K,
    init       = "k-means++",
    n_init     = 10,
    max_iter   = 300,
    random_state = 42,
)

# fit_predict() → o'rgatib, har nuqta uchun klaster raqami qaytaradi
# labels_ → [0, 2, 1, 0, 2, ...] – har nuqta qaysi klasterda
labels = kmeans.fit_predict(X)

# cluster_centers_ → har klasterning markaziy koordinatalari (K×2)
markazlar = kmeans.cluster_centers_

# inertia_ → har nuqtadan o'z klaster markazigacha
#           kvadrat masofalar yig'indisi (WCSS)

```

```

#           Kichik → klasterlar ixchamroq
wcss = kmeans.inertia_

# Silhouette score → klasterlar sifatini o'lchaydi
# -1 dan +1 gacha: +1 → mukammal, 0 → chalkash, -1 → noto'g'ri
silhouette = silhouette_score(X, labels)

print("\n" + "=" * 52)
print("  K-MEANS NATIJALARI:")
print("=" * 52)
print(f"  Klasterlar soni : {K} ta")
print(f"  WCSS (Inertia)  : {wcss:.4f}")
print(f"  Silhouette Score: {silhouette:.4f}  "
      f"({'Yaxshi ✓' if silhouette > 0.5 else 'O`rtacha'})")

print(f"\n  Klaster markazlari:")
print(f"  {'Klaster':<10} {'X markaz':>12} {'Y markaz':>12}  "
      f"{'Nuqtalar':>10}")
print("  " + "-" * 46)
for k in range(K):
    son = np.sum(labels == k)
    print(f"  Klaster {k}  "
          f"  {markazlar[k,0]:>10.4f}"
          f"  {markazlar[k,1]:>10.4f}"
          f"  {son:>10} ta")

# =====
# 3. ELBOW USULI – OPTIMAL K TOPISH
# =====

# Elbow (tirsak) usuli:
# K=1 dan K=9 gacha WCSS ni hisoblab, grafikda ko'rsatish
# WCSS keskin tushishdan to'xtagan nuqta → optimal K
k_qiymatlar = range(1, 10)
wcss_royxat = []

for k in k_qiymatlar:
    km = KMeans(
        n_clusters = k,
        init        = "k-means++",
        n_init      = 10,
        random_state = 42,
    )
    km.fit(X)
    wcss_royxat.append(km.inertia_)

# =====
# 4. VIZUALIZATSIYA – 3 ta grafik
# =====

fig = plt.figure(figsize=(16, 5), facecolor="#0f1117")
gs  = gridspec.GridSpec(1, 3, figure=fig, wspace=0.38)

RANGLAR    = ["#7c85f3", "#f38ba8", "#a6e3a1"]  # Klaster ranglari
MARKER_SZ  = 65  # Nuqta o'lchami

# — 4a. Dastlabki (klasterlashdan oldin) —————

```

```

ax1 = fig.add_subplot(gs[0])
ax1.set_facecolor("#1a1d27")

# Barcha nuqtalar bir rangda – hech qanday klaster yo'q
ax1.scatter(
    X[:, 0], X[:, 1],
    color      = "#8892b0",
    s          = MARKER_SZ,
    alpha      = 0.65,
    edgecolors = "#45475a",
    linewidths = 0.5,
    zorder     = 3,
    label      = f"{len(X)} ta nuqta",
)

ax1.set_title(
    "Dastlabki Ma'lumot\n(Klasterlashdan oldin)",
    color="#e2e8f0", fontsize=12,
    fontweight="bold", pad=12,
)
ax1.set_xlabel("X", color="#8892b0", fontsize=11)
ax1.set_ylabel("Y", color="#8892b0", fontsize=11)
ax1.legend(
    fontsize=9, facecolor="#252836",
    edgecolor="#363a52", labelcolor="#cdd6f4",
)
ax1.grid(color="#2d3148", linestyle="--", alpha=0.4, zorder=1)
ax1.tick_params(colors="#8892b0")
for spine in ax1.spines.values():
    spine.set_edgecolor("#2d3148")

```

— 4b. K-means natijasi (asosiy grafik) —————

```

ax2 = fig.add_subplot(gs[1])
ax2.set_facecolor("#1a1d27")

# Har klasterning nuqtalarini alohida rangda chizish
for k in range(K):
    # Boolean masking → faqat k-klasterdagi nuqtalar
    maska      = labels == k
    nuqtalar   = X[maska]

    ax2.scatter(
        nuqtalar[:, 0], nuqtalar[:, 1],
        color      = RANGLAR[k],
        s          = MARKER_SZ,
        alpha      = 0.70,
        edgecolors = "#0f1117",
        linewidths = 0.5,
        zorder     = 3,
        label      = f"Klaster {k} ({maska.sum()} ta)",
    )

# Klaster markazlarini yulduz belgisi bilan ko'rsatish
ax2.scatter(
    markazlar[:, 0], markazlar[:, 1],

```

```

        color      = "#fab387",
        s          = 280,
        marker     = "*",          # Yulduz marker
        edgecolors = "#0f1117",
        linewidths = 1.2,
        zorder     = 5,
        label      = "Markazlar (★)",
    )

    # Har markazga koordinatalarini yozish
    for k, (mx, my) in enumerate(markazlar):
        ax2.annotate(
            f"({mx:.1f}, {my:.1f})",
            xy          = (mx, my),
            xytext      = (10, 10),
            textcoords  = "offset points",
            color       = "#fab387",
            fontsize    = 8.5,
            fontweight  = "bold",
            zorder      = 6,
        )

    ax2.set_title(
        f"K-Means Klasterlash (K={K})\n"
        f"Silhouette = {silhouette:.3f}",
        color="#e2e8f0", fontsize=12,
        fontweight="bold", pad=12,
    )
    ax2.set_xlabel("X", color="#8892b0", fontsize=11)
    ax2.set_ylabel("Y", color="#8892b0", fontsize=11)
    ax2.legend(
        fontsize=9, facecolor="#252836",
        edgecolor="#363a52", labelcolor="#cdd6f4",
    )
    ax2.grid(color="#2d3148", linestyle="--", alpha=0.4, zorder=1)
    ax2.tick_params(colors="#8892b0")
    for spine in ax2.spines.values():
        spine.set_edgecolor("#2d3148")

```

— 4c. Elbow grafigi —————

```

ax3 = fig.add_subplot(gs[2])
ax3.set_facecolor("#1a1d27")

```

WCSS chiziq

```

ax3.plot(
    k_qiyamatlar, wcss_royxat,
    color      = "#7c85f3",
    linewidth  = 2.5,
    marker     = "o",
    markersize = 7,
    zorder     = 3,
    label      = "WCSS",
)

```

Optimal K ni alohida belgilash

```

ax3.scatter(

```

```

K, wcss_royxat[K - 1],
color      = "#f38ba8",
s          = 180,
zorder     = 5,
edgecolors = "#0f1117",
linewidths = 1.5,
label      = f"Optimal K={K}",
)

# Optimal K uchun vertikal chiziq
ax3.axvline(
    x          = K,
    color      = "#f38ba8",
    linewidth  = 1.5,
    linestyle  = "--",
    alpha      = 0.7,
    zorder     = 2,
)

# Har nuqtaga WCSS qiymati yozish
for k, wcss_val in zip(k_qiymatlar, wcss_royxat):
    ax3.annotate(
        f"{wcss_val:.0f}",
        xy      = (k, wcss_val),
        xytext  = (0, 10),
        textcoords = "offset points",
        ha      = "center",
        color    = "#8892b0",
        fontsize = 8,
    )

ax3.set_xticks(list(k_qiymatlar))
ax3.set_xlabel("K (Klasterlar soni)",
               color="#8892b0", fontsize=11, labelpad=8)
ax3.set_ylabel("WCSS (Inertia)",
               color="#8892b0", fontsize=11, labelpad=8)
ax3.set_title(
    "Elbow Usuli\n(Optimal K topish)",
    color="#e2e8f0", fontsize=12,
    fontweight="bold", pad=12,
)
ax3.legend(
    fontsize=9, facecolor="#252836",
    edgecolor="#363a52", labelcolor="#cdd6f4",
)
ax3.grid(color="#2d3148", linestyle="--", alpha=0.4, zorder=1)
ax3.tick_params(colors="#8892b0")
for spine in ax3.spines.values():
    spine.set_edgecolor("#2d3148")

plt.suptitle(
    "K-Means Klasterlash – 2D Tasodifiy Nuqtalar",
    fontsize=15, fontweight="bold",
    color="#e2e8f0", y=1.02,
)

```

```
plt.tight_layout()
plt.show()
```

Chiqish Natijasi

```
=====
MA'LUMOT GENERATSIYASI:
=====
Jami nuqtalar : 105 ta
O'lchamlar    : 2 ta (X, Y)
X oralig'i    : [-5.71, 5.82]
Y oralig'i    : [-5.63, 6.71]

=====
K-MEANS NATIJALARI:
=====
Klasterlar soni : 3 ta
WCSS (Inertia)  : 89.3241
Silhouette Score: 0.7134 (Yaxshi ✓)

Klaster markazlari:
Klaster      X markaz      Y markaz      Nuqtalar
-----
Klaster 0    -2.9841      -3.0217      35 ta
Klaster 1     0.0523       3.9814      35 ta
Klaster 2     3.0192      -1.9876      35 ta
```

Dastur Tuzilishi

```
Dastur
├── Ma'lumot generatsiya
│   ├── np.random.randn() → normal taqsimlangan nuqtalar
│   ├── * tarqalish       → kenglik sozlash
│   └── np.vstack()       → 3 guruhni birlashtirish
├── K-Means modeli
│   ├── KMeans(k-means++) → aqlli boshlang'ich tanlash
│   ├── fit_predict()     → o'rgatish + klaster belgilash
│   └── cluster_centers_  → markazlar koordinatalari
├── Sifat baholash
│   ├── inertia_ (WCSS)   → ixchamlik o'lchovi
│   └── silhouette_score() → ajralish sifati
├── Elbow usuli
│   └── K=1..9 WCSS       → optimal K vizual topish
└── Vizualizatsiya
    ├── scatter (oldin)   → klasterlashdan oldin
    ├── scatter (keyin)   → rang + markazlar
    └── plot (elbow)      → WCSS vs K grafigi
```

Asosiy Texnik Tushunchalar

K-MEANS ALGORITMI – QANDAY ISHLAYDI?

- 1-qadam: K ta markaz tasodifiy joylashtiriladi
(k-means++ → aqlliyoq tanlaydi)
- 2-qadam: Har nuqta eng yaqin markazga biriktiriladi
 $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ ← Evklid masofasi
- 3-qadam: Har klasterning yangi markazi hisoblanadi
markaz = nuqtalar o'rtachasi
- 4-qadam: Markazlar o'zgarmasa → TO'XTASH
O'zgansa → 2-qadam ga qaytish

Vizual:

Boshlang'ich:	1-iteratsiya:	Yakuniy:
x . . x	● - . - x	● . . .
. . x .	. . ● - .	 ●
. x . .	. x . .	. . ● .
(x=markaz)	(yangi markaz)	(barqaror)

WCSS vs SILHOUETTE:

WCSS (Inertia):

$$\sum \sum ||x_i - \mu_k||^2$$

Kichik → nuqtalar markazga yaqin → ixcham klaster

Kamchi: K oshsa doim kamayadi → yolg'on signal

Faqat ELBOW grafigi bilan ma'noga ega

Silhouette Score:

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

a(i) → o'z klasteridagi o'rtacha masofa

b(i) → eng yaqin boshqa klaster o'rtacha masofasi

+1.0 → mukammal ajralish

0.0 → klasterlar bir-biriga tegib turibdi

-1.0 → nuqta noto'g'ri klasterda

K-MEANS++ vs RANDOM:

RANDOM:	K-MEANS++:
x x x	x
(uchala bir	(1-markaz istalgan joyga)
burchakda	x
bo'lishi mumkin)	(2-markaz uzoqroqqa)
	x
	(3-markaz yanada uzoqroqqa)
Muammo: local	→ Global minimumga yaqinroq
minimum tuzog'i	→ Tezroq konvergensiya

ELBOW USULI:

K=1 → WCSS=850 (barcha nuqta 1 markazda)
K=2 → WCSS=420 (keskin tushdi!)
K=3 → WCSS=89 (yana keskin!) ← TIRSAK = optimal K
K=4 → WCSS=75 (sekin tushdi)
K=5 → WCSS=68 (deyarli o'zgarmadi)
Tirsak nuqtasi → qo'shimcha K uncha foyda bermaydi

💡 **Eslab qol:** K-Means **doira shaklidagi** klasterlarni yaxshi topadi, lekin **o'roq yoki oy shaklida** joylashgan nuqtalar uchun yomon ishlaydi. Bunday hollarda **DBSCAN** (zichlikka asoslangan) yoki **Gaussian Mixture Models** ishlatiladi. Shuningdek, K-means **outlier larga sezgir** — bitta chekka nuqta markaz hisoblashni buzishi mumkin. Yechim: klasterlashdan oldin **StandardScaler** bilan masshtablash va outlierlarni olib tashlash.

● **SAVOL 58:**

[DECISION TREE] **DECISION TREE KLASSIFIKATORI O'RGATING. MAX_DEPTH=3 VA MAX_DEPTH=10 UCHUN TRAIN/TEST ACCURACY NI SOLISHTIRING VA OVERFITTING NI GRAFIK ORQALI KO'RSATING.**

1. Decision Tree Va "max_depth" Tushunchasi

Decision Tree (Qarorlar daraxti):

Ma'lumotlarni muayyan shartlar asosida shoxlarga ajratib boruvchi algoritm.

Har bir tugun (node) bitta xususiyatni (feature) tekshiradi.

max_depth (Daraxtning maksimal chuqurligi):

Daraxtning ildizidan (root) eng oxirgi bargigacha (leaf) bo'lgan qadamlar soni.

Bu giperparametr modelning qanchalik murakkab bo'lishini belgilaydi.

Overfitting (O'ta moslashish):

Modelning o'quv ma'lumotlarini (train data) yodlab olishi, lekin yangi, ko'rilmagan ma'lumotlarda (test data) xato ishlashi.

max_depth qanday ta'sir qiladi?

max_depth = 3 (Sayoz daraxt):

- Model sodda bo'ladi.
- Asosiy qonuniyatlarni o'rganadi.
- Generalization (umumlashtirish) yaxshi bo'ladi.

max_depth = 10 (Chuqur daraxt):

- Model juda murakkablashadi.
- Ma'lumotdagi shovqinlarni (noise) ham o'rganib oladi.
- Train accuracy deyarli 100% bo'ladi, lekin Test accuracy tushib ketadi.
- Natija: OVERFITTING!

2. Ma'lumotlarni Tayyorlash Va Modelni O'rgatish

Quyida sintetik ma'lumotlar to'plami yaratib, **max_depth=3** va **max_depth=10** uchun alohida modellar o'rgatamiz.


```

# Kerakli kutubxonalarni yuklab olamiz
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Sintetik ma'lumotlar to'plamini yaratamiz
# 1000 ta namuna, 20 ta xususiyat (shundan 5 tasi informativ, qolgani shovqin)
X, y = make_classification(n_samples=1000, n_features=20, n_informative=5,
                           n_redundant=2, random_state=42)

# 2. Ma'lumotlarni Train (80%) va Test (20%) qismlarga ajratamiz
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# -----
# 3. Model 1: Sayoz daraxt (max_depth=3)
# -----
tree_depth_3 = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_depth_3.fit(X_train, y_train)

# Natijalarni hisoblaymiz
train_acc_3 = accuracy_score(y_train, tree_depth_3.predict(X_train))
test_acc_3 = accuracy_score(y_test, tree_depth_3.predict(X_test))

# -----
# 4. Model 2: Chuqur daraxt (max_depth=10)
# -----
tree_depth_10 = DecisionTreeClassifier(max_depth=10, random_state=42)
tree_depth_10.fit(X_train, y_train)

# Natijalarni hisoblaymiz
train_acc_10 = accuracy_score(y_train, tree_depth_10.predict(X_train))
test_acc_10 = accuracy_score(y_test, tree_depth_10.predict(X_test))

# Natijalarni ekranga chiqarish
print(f"--- max_depth=3 ---")
print(f"Train Accuracy: {train_acc_3:.4f}")
print(f"Test Accuracy : {test_acc_3:.4f}\n")

print(f"--- max_depth=10 ---")
print(f"Train Accuracy: {train_acc_10:.4f}")
print(f"Test Accuracy : {test_acc_10:.4f}")

```

Kutilayotgan Natija (Output):

```

--- max_depth=3 ---
Train Accuracy: 0.8650
Test Accuracy : 0.8500

--- max_depth=10 ---
Train Accuracy: 0.9925
Test Accuracy : 0.8150

```

Izoh: `max_depth=10` da Train Accuracy deyarli 100% ga (0.9925) yetdi, ya'ni model test savollarini "yodlab" oldi. Ammo Test Accuracy 0.8150 gacha tushib ketdi. Bu ochiq-oydin **Overfitting** belgisidir.

3. Overfitting Ni Grafik Orqali Ko'rsatish

Faqat ikkita chuqurlik emas, balki 1 dan 15 gacha bo'lgan barcha chuqurliklar uchun modelni sinab ko'ramiz va natijani vizualizatsiya qilamiz.

```
# Chuqurliklar ro'yxati (1 dan 15 gacha)
depths = range(1, 16)
train_scores = []
test_scores = []

# Har bir chuqurlik uchun modelni o'rgatamiz va natijalarni saqlaymiz
for depth in depths:
    # Modelni yaratish va o'rgatish
    clf = DecisionTreeClassifier(max_depth=depth, random_state=42)
    clf.fit(X_train, y_train)

    # Train va Test xatoliklarni ro'yxatga qo'shish
    train_scores.append(accuracy_score(y_train, clf.predict(X_train)))
    test_scores.append(accuracy_score(y_test, clf.predict(X_test)))

# -----
# Grafik chizish
# -----
plt.figure(figsize=(10, 6))

# Train accuracy - Ko'k rangda
plt.plot(depths, train_scores, label='Train Accuracy', marker='o', color='blue')

# Test accuracy - Qizil rangda
plt.plot(depths, test_scores, label='Test Accuracy', marker='s', color='red')

plt.title('Decision Tree: Overfitting ni kuzatish (max_depth ta\'siri)',
          fontsize=14)
plt.xlabel('max_depth (Daraxt chuqurligi)', fontsize=12)
plt.ylabel('Accuracy (Aniqlik)', fontsize=12)

# Optimal nuqtani (max_depth=3 yoki 4) ko'rsatuvchi chiziq
plt.axvline(x=4, color='gray', linestyle='--', label='Optimal Nuqta')

plt.xticks(depths)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Grafikning tahlili (Nima sodir bo'lyapti?):

X-o'qi: max_depth (1 dan 15 gacha)
Y-o'qi: Accuracy (0.0 dan 1.0 gacha)

- Ko'k chiziq (Train Accuracy):
Daraxt chuqurlashgan sari muntazam o'sib boradi va oxiri 1.0 (100%) ga yetadi.

Model o'ziga berilgan ma'lumotlarni mukammal "yodlab" oladi.

● Qizil chiziq (Test Accuracy):

Dastlab ko'k chiziq bilan birga o'sadi (Underfitting dan qutulish bosqichi).

Chuqurlik 4 ga yetganda o'zining eng yuqori cho'qqisiga chiqadi.

Undan keyin (depth 5, 6, 7... 15) u tushishni boshlaydi.

Kesisish/Ajralish nuqtasi:

Ko'k chiziq yuqoriga, qizil chiziq pastga ketayotgan oraliq (depth > 4) anyan OVERFITTING hududi hisoblanadi.

4. Xulosa

`max_depth = 3:`

- ✓ Model qonuniyatni to'g'ri angladi (Sweet spot).
- ✓ Train va Test o'rtasidagi farq kichik.
- ✓ Yangi ma'lumotlarda yaxshiroq ishlaydi.

`max_depth = 10:`

- ✗ Model ma'lumotdagi "shovqin"larni xususiyat sifatida qabul qildi.
- ✗ O'quv ma'lumotlarini 100% ga yaqin yodladi.
- ✗ Test ma'lumotlarida past natija ko'rsatdi (Overfitting).

Yechimlar:

1. Daraxtni kesish (Pruning): `max_depth`, `min_samples_split`, `min_samples_leaf` kabi giperparametrlarni cheklash.
2. Ansambl usullari: Bitta chuqur daraxt o'rniga Random Forest kabi ko'plab sayoz/har xil daraxtlardan foydalanish.

💡 **Eslab qol:** Decision Tree tabiatan har doim ma'lumotlarni 100% ajratguncha o'sishga moyil bo'ladi. Agar siz daraxtning o'sishini qattiq nazorat qilmasangiz (masalan, `max_depth` orqali chegara qo'ymasangiz), u har doim o'quv ma'lumotlarini yodlab olib, overfitting botqog'iga tushib qoladi. Yaxshi model imtihon savollarini yodlagan emas, balki fanni tushungan talabaga o'xshashi kerak!

● **SAVOL 59:**

[NUMPY + MATPLOTLIB] SIN VA COS FUNKSIYALARINI $[0, 4\pi]$ ORALIQDA 200 NUQTA BILAN HISOBLANG VA IKKALASINI BIR GRAFIKDA TURLI RANG VA LABEL BILAN KO'RSATING. GRID VA LEGEND QO'SHING.

1. Masalaning Mohiyati Va Kutubxonalar

Ushbu vazifani bajarish uchun bizga Python ekotizimidagi ikkita eng asosiy kutubxona kerak bo'ladi:

- **NumPy** : Raqamli hisoblashlar va massivlar (array) bilan ishlash uchun. U yordamida x o'qidagi nuqtalarni va ularning trigonometrik qiymatlarini ($\sin$ va $\cos$) juda tez hisoblaymiz.
 - **Matplotlib** : Hisoblangan ma'lumotlarni vizualizatsiya qilish, ya'ni chiroyli va tushunarli grafik chizish uchun.
-

2. Dastur Kodi Va Vizualizatsiya

Quyida so'ralgan vazifani to'liq bajaruvchi Python kodi keltirilgan:

```
# Kerakli kutubxonalarni chaqirib olamiz
import numpy as np
import matplotlib.pyplot as plt

# 1. x o'qi uchun ma'lumotlarni tayyorlash
# 0 dan 4*pi gacha bo'lgan oraliqda bir xil masofada joylashgan 200 ta nuqta
yaratamiz
x = np.linspace(0, 4 * np.pi, 200)

# 2. y o'qi uchun sin(x) va cos(x) qiymatlarini hisoblash
y_sin = np.sin(x)
y_cos = np.cos(x)

# 3. Grafik chizish qismi
plt.figure(figsize=(10, 5)) # Grafik o'lchamini belgilash (kenglik=10, balandlik=5)

# Sinus grafigini chizish (ko'k rang, qalinligi 2)
plt.plot(x, y_sin, color='blue', linewidth=2, label='sin(x)')

# Kosinus grafigini chizish (qizil rang, qalinligi 2, chiziq uslubi uzuq-uzuq)
plt.plot(x, y_cos, color='red', linewidth=2, linestyle='--', label='cos(x)')

# 4. Grafikni bezash va qo'shimcha elementlar
plt.title('Sinus va Kosinus Funksiyalarining Grafigi', fontsize=14,
fontweight='bold')
plt.xlabel('x qiymatlari (0 dan 4π gacha)', fontsize=12)
plt.ylabel('y qiymatlari (Amplitude)', fontsize=12)

# Grid (to'r) qo'shish
plt.grid(True, linestyle=':', alpha=0.7)

# Legend (shartli belgilar) qo'shish
plt.legend(loc='upper right', fontsize=12)

# Grafikni ekranga chiqarish
plt.show()
```

3. Kodning Har Bir Qadamiga Izoh

- `np.linspace(start, stop, num)` : Bu funksiya berilgan oraliqni (`start` dan `stop` gacha) `num` ta teng qismlarga bo'lib beradi. Bizning holatda 0 dan 4π gacha 200 ta nuqta yaratildi. Nuqtalar soni qancha ko'p bo'lsa (masalan 200 ta), grafik shunchalik silliq (smooth) chiqadi.

- `np.pi` : π (Pi) ning matematik qiymati (3.1415...).
- `np.sin()` va `np.cos()` : NumPy ning afzalligi shundaki, bu funksiyalarga butun boshli massivni (`x`) bersangiz, u tsikl (for-loop) ishlatmasdan birdaniga barcha 200 ta nuqta uchun javobni hisoblab, yangi massiv qaytaradi (vektorizatsiya).
- `plt.plot(x, y, ...)` : `x` va `y` nuqtalarni tutashtirib chiziqli grafik chizadi.
- `color` va `label` : Har bir chiziqqa alohida rang va nom berdik. Nom (`label`) keyinchalik `plt.legend()` ishlaganda ajratib ko'rsatish uchun kerak.
- `plt.grid(True)` : Grafik orqasiga yordamchi katakchalar (to'r) chizadi. Bu nuqtalarning aniq qayerda joylashganini ko'z bilan chamalashni osonlashtiradi.
- `plt.legend()` : Burchakda "Ko'k chiziq - $\sin(x)$, Qizil chiziq - $\cos(x)$ " degan ma'lumot qutisini chiqaradi.

💡 **Eslab qol:** Data Science va ma'lumotlar tahlilida **NumPy** bu sizning "miyangingiz" (hisoblashlarni tez va samarali bajaradi), **Matplotlib** esa sizning "tilingiz/ko'zingiz" dir (shu hisob-kitoblarni boshqalarga tushunarli qilib ko'rsatib beradi). Ularning birgalikdagi ishlashi har qanday tahlilning ajralmas qismidir.

● SAVOL 60:

[SKLEARN PIPELINE] STANDARDSCALER VA MLPCLASSIFIER DAN IBORAT PIPELINE TUZING. SKLEARN.DATASETS DAGI WINE, BREAST_CANCER YOKI DIGITS DATASET USTIDA O'RGATING VA CROSS_VAL_SCORE BILAN BAHOLANG.

1. Asosiy Tushunchalar: Pipeline Va MLPClassifier

Pipeline (Konveyer):

Ma'lumotlarni qayta ishlash (preprocessing) va modelga berish qadamlarini bitta yaxlit jarayonga aylantiruvchi vosita.
Bu kodni toza saqlaydi va eng muhimi – Data Leakage (ma'lumot sizishi) ning oldini oladi.

StandardScaler:

Ma'lumotlarni standartlashtiradi.
Formula: $z = (x - \mu) / \sigma$
(μ - o'rtacha qiymat, σ - standart og'ish).
Natijada barcha xususiyatlar (features) bir xil masshtabga keladi.

MLPClassifier (Multi-Layer Perceptron):

Scikit-learn tarkibidagi klassik sun'iy neyron tarmoq modeli.
Neyron tarmoqlar gradient tushish (Gradient Descent) algoritmidan foydalanganligi sababli, ularga berilayotgan ma'lumotlar ALBATTA standartlashtirilgan bo'lishi kerak. Aks holda model yaxshi o'rganmaydi.

`cross_val_score` (K-Fold Cross-Validation):

Modelni bir marta emas, balki bir necha bor turli xil train/test qismlarda sinab ko'rish. Bu modelning barqarorligini ko'rsatadi.

2. Python Kod: Pipeline Va Cross-Validation

Quyida `breast_cancer` (ko'krak bezi saratoni) ma'lumotlar to'plami ustida Pipeline qurib, uni 5 qismli (5-fold) cross-validation yordamida baholaymiz.

```
# Kerakli kutubxonalarni yuklab olamiz
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import cross_val_score

# 1. Ma'lumotlarni yuklash (Breast Cancer dataset)
data = load_breast_cancer()
X = data.data
y = data.target

# 2. Pipeline yaratish
# Ketma-ketlik: avval 'scaler', keyin 'mlp' modeli
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    # max_iter=1000 qilib belgilaymiz, aks holda model convergence (yetib borish)
    # xatosini berishi mumkin
    ('mlp', MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000,
random_state=42))
])

# 3. Modelni Cross-Validation yordamida baholash
# cv=5 degani ma'lumotni 5 qismga bo'lib, 5 marta o'qitib sinaydi
scores = cross_val_score(pipeline, X, y, cv=5, scoring='accuracy')

# 4. Natijalarni chop etish
print("Har bir fold (qism) uchun aniqlik (Accuracy):")
print(np.round(scores, 4))
print("-" * 40)
print(f"O'rtacha aniqlik (Mean Accuracy): {scores.mean():.4f}")
print(f"Standart og'ish (Standard Deviation): {scores.std():.4f}")
```

Kutilayotgan Natija (Output):

```
Har bir fold (qism) uchun aniqlik (Accuracy):
[0.9825 0.9737 0.9737 0.9825 0.9912]
-----
O'rtacha aniqlik (Mean Accuracy): 0.9807
Standart og'ish (Standard Deviation): 0.0066
```

Izoh: Model o'rtacha **98%** aniqlikda ishlamoqda. Standart og'ish juda kichik (0.0066), demak modelimiz barqaror va test qismlari o'zgarganda xatolik ko'p sakrab ketmayapti.

3. Nima Uchun Pipeline Ishlatish Shart? (Data Leakage Muammosi)

Tasavvur qiling, siz Pipeline ishlatmasdan ma'lumotlarni shunday tahlil qildingiz:

1. Butun `X` ma'lumotni `StandardScaler` dan o'tkazdingiz.
2. Keyin `cross_val_score` yordamida baholadingiz.

Bu xato yondashuv! Sababi `StandardScaler` butun ma'lumotning o'rtacha qiymatini hisoblaganda, hali yashirin bo'lishi kerak bo'lgan "Test" qismidagi ma'lumotlarni ham ko'rib qo'yadi. Bu **Data Leakage (Ma'lumot sizib chiqishi)** deyiladi. Model hayotda yo'q yuqori natijani ko'rsatib, sizni aldaydi.

Pipeline qanday ishlaydi: Pipeline ma'lumotni 5 qismga bo'ladi. Har safar 4 ta qismni olib, **FAQAT shu 4 ta qism asosida** `StandardScaler` ni moslaydi (`fit`) va modelni o'rgatadi. Qolgan 1 ta qismni esa faqat `transform` qilib, testdan o'tkazadi. Shunday qilib, test ma'lumotlari haqiqiy "ko'rilmagan" ma'lumot bo'lib qoladi.

💡 **Eslab qol:** Sun'iy neyron tarmoqlar (MLP, Deep Learning) ma'lumotlar o'lchamiga juda ta'sirchan. Bitta xususiyat minglarda, ikkinchisi o'nliklarda bo'lsa, gradientlar portlab ketishi yoki juda sekin ishlashi mumkin. Shuning uchun `StandardScaler` + `MLP` eng yaxshi do'stlardir. Pipeline orqali ularni doim birga olib yuring.

Endi ushbu Pipeline ichidagi `MLPClassifier`'ning eng maqbul giperparametrlarini (masalan, yashirin qatlamlar soni yoki *learning rate*) topish uchun `GridSearchCV` ni qanday qo'llashni ko'rib chiqishni xohlaysizmi?

In []: